A Mesh-Network Simulation Framework with CRDTs

Gabriela Rodrigues da Silva

Introduction

Context

Problem Description

1. Many distributed systems assume direct connections between replicas.
    - *What if direct links are not available?*
2. Routing algorithms and other mesh networking principles can be used to take advantage of indirect paths.
3. Operation-based CRDTs introduce the added challenge of requiring causal delivery.

Objectives

- Create a simulated network environment.
- Test operation-based CRDT operations in the context of changing network topologies.
- Compare different routing protocols.

Background

CRDTs

- Abstract distributed data types with useful properties for preserving availability:
  - Independent concurrent updates
  - Deterministic merge operations with conflict resolution
  - Guaranteed eventual consistency
- State-based CRDTs frequently involve propagating large amounts of data, while operation-based CRDTs reduce communication volume but may require causal delivery.

Routing Algorithms

- **Flooding:** Each node forwards received messages to all neighbours except the sender, once.
- **Distance-Vector:** Each node maintains a routing table of distances and next hops to each destination.
- **Link-State:** Each node stores a network graph and independently calculates the best next hop.

Solution

Simulation Framework



- Written in C#.
  - SimSharp (SimPy port) for discrete event handling.
- Modular application architecture supporting three different routing algorithms over both simulated and real-time underlying network communication.
- Small operation-based CRDT library.
- Benchmark application behaviours and experiments.

Visualisation Tool

- Written for WPF
- Allows for the creation of arbitrary network topologies.
- Displays message sending patterns in real-time.

Implementation Details

CRDTs

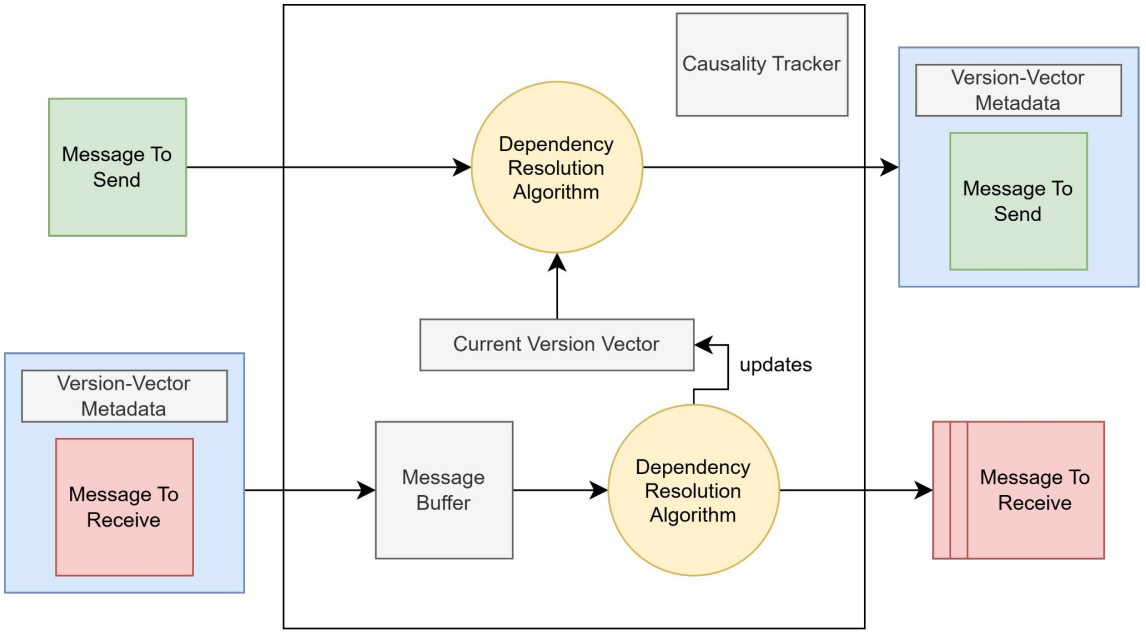
Note: ORSet insertion operations were used for benchmarking due to the relative complexity of this data type.

GCounter	Per-replica increment counters; value is their sum. Messages carry only the updated replica's value.
PNCounter	Two GCounters (increments, decrements); value is increments minus decrements. Messages carry only the changed value.
GSet	Wrapper around a HashSet; merge is set-union. No removals.
ORSet:	Keeps a set of insert tags and a set of remove tags; an item is present if it has an insert tag not in the removed set. Merge unions both sets. Insert messages carry the value and its tag; remove messages carry all observed tags for the value.
LWWRegister	Stores a (timestamp, value) pair; merge keeps the most recent timestamp.
MVRegister	Stores (version vector, value) pairs; keeps all concurrent values and discards them once a causally newer value is seen.

Causal Delivery

Version-Vector-Based Messaging Middleware

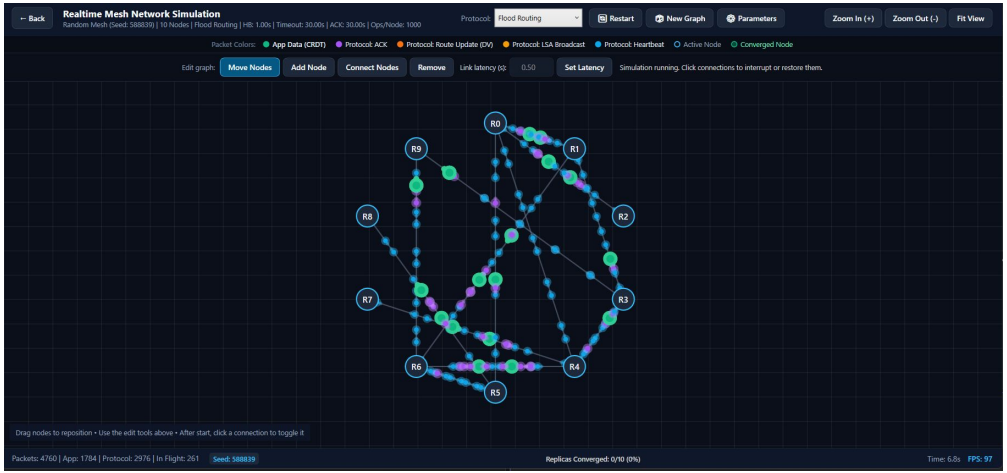
- Wraps outgoing messages with causal metadata.
- Relies on a message buffering system to await missing dependencies.



Visualization Tool

Features

- Preset and custom topologies
- Interactive graph editor
  - Creating, removing and connecting nodes;
  - Connecting and disconnecting nodes in real-time
  - Per-link latency configuration
- Real-time packet animation and color-coding



Routing Algorithms

Flooding

- Every node forwards each message to all active neighbours, with de-duplication tags and a hop limit
- No routing state and no setup time, so it is the simplest and most robust option
- Sent the most messages in every topology and scaled worst, especially in dense networks
- Placed a heavy load on the host machine and occasionally timed out in large networks

Distance Vector

- Nodes periodically send their distance tables to neighbours, and a distributed Bellman-Ford finds the next hop
- Fewer messages than flooding, but the slowest to complete (20.7 s in the random topology baseline, against 0.56 s for flooding and 0.87 s for link-state)
- The delay comes from routing information propagating slowly, plus timeouts
- The most sensitive to its own timeout parameters

Link-State

- Nodes flood link-state advertisements (LSAs) at startup and on topology changes, then run Dijkstra locally
- Fewest messages in every topology except the complete graph
- Fast convergence, because changes propagate quickly
- Message count grows with the number of edges

Experiments

Description

Basis:

- Insertion operations in an ORSet. Each replica was assigned a number of different elements and simulations finish when all replicas have the total number of elements in their respective ORSets.
- Runs without routing algorithms were performed for comparison, but only terminated successfully in complete graphs.

Topologies:

- Complete graph
- Star
- Ring
- Erdős-Rényi network
- Barabási-Albert network

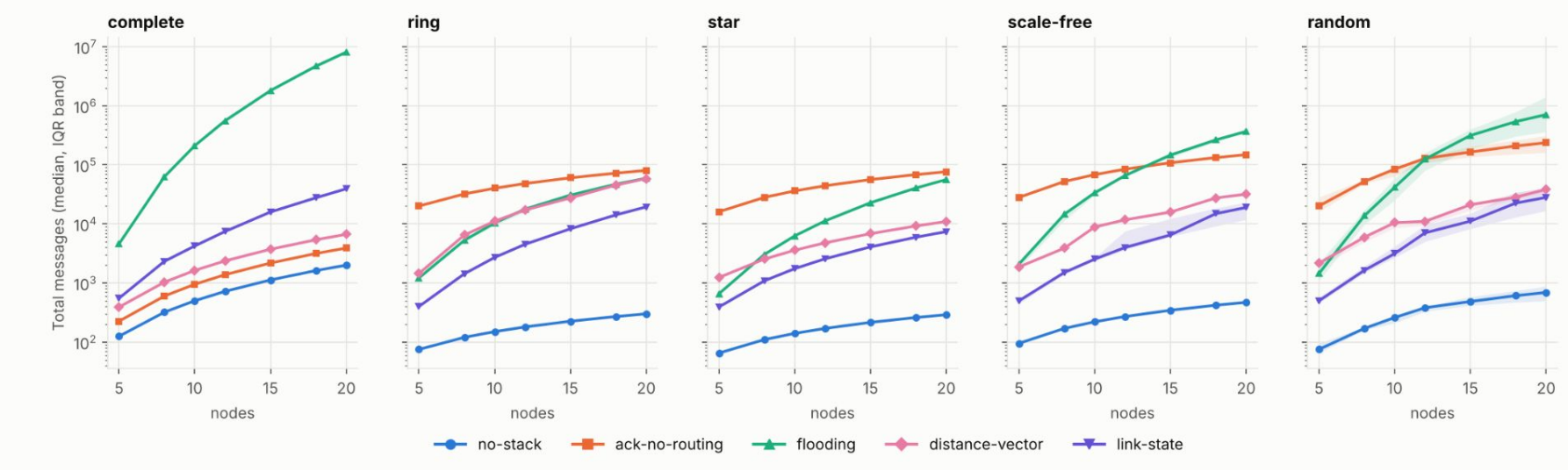
Parameter Variations:

- Number of replicas.
- Communication latency.
- Algorithm-specific parameters.

Metrics:

- Execution time.
- Number of messages before and after convergence.

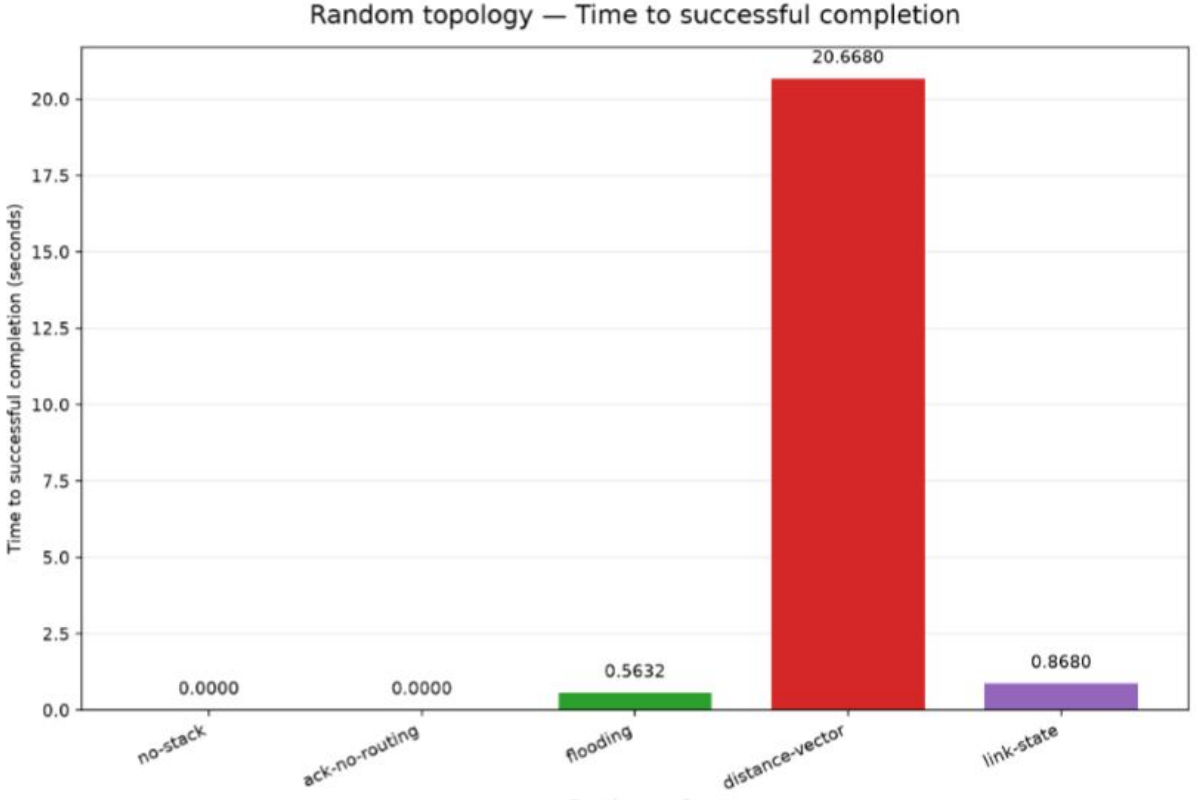
Total Messages Sent Per Number of Nodes



Execution Time

The distance-vector algorithm took longer to complete due to routing information propagation time.

The graph refers to a baseline with 10 replicas and link delays between 0.01 and 0.2 time units.



Methodological Limitations

- Causal delivery waits for all predecessors implied by a version vector, which is sometimes unnecessary when the final result already dominates them
- Discrete-event simulation assumes computation time is negligible compared with network time
- Routing algorithms kept in their simplest form because of assumptions about replicas' mutual knowledge (real protocols like OSPF are more complex)
- JSON encoding is verbose; a more compact serialisation format could shrink messages
- The more linear simulation execution may hide concurrency or parallelism bugs

Scope Limitations

- Limited set of CRDTs and test benchmarks
- Dynamic network behaviour (node disconnection, variable latency) not fully explored
- No simulations with larger numbers of replicas, due to limited computational power (especially for flooding)
- Visualisation tool is Windows-only

Conclusions

Conclusions

- Routing enables mesh delivery. All algorithms succeeded in nearly every configuration.
- Specialised routing beats flooding. Flooding sent the most messages and scaled poorly.
- Link-state performs best overall. It minimised messages and completed quickly.
- Distance-vector suits dense networks. It runs slower but scales better in message count as density increases.
- Causal delivery preserved. Version vectors and buffering kept CRDTs correct across all topologies.
- Sparse networks work efficiently. Routing enabled convergence in ring and star topologies with modest overhead.
- Framework delivered: A modular C# simulator, CRDT library and network visualisation tool.